Mathematical Foundation of Computer Science

Dr. Mukesh Kumar
Associate Professor
Department of Engineering & Technology
Gurugram University, Gurugram, Haryana

2026

Contents

1 Set Theory and Logic **7**
- 1.1 Set Theory Fundamentals 7
- 1.2 Set Operations 7
- 1.3 Inclusion-Exclusion Principle 8
- 1.4 Relations 8
  - 1.4.1 Relation Properties 8
- 1.5 Equivalence Relations 8
- 1.6 Partial Orders and Hasse Diagrams 9
- 1.7 Functions 9
- 1.8 Cardinality 10
- 1.9 Propositional Logic 10
- 1.10 Tautologies and Satisfiability 11
- 1.11 Logical Equivalences 11
- 1.12 Normal Forms 11

2 Predicate Logic and Proof Techniques **13**
- 2.1 Predicate Logic Syntax 13
- 2.2 Valid Inference Rules 13
- 2.3 Proof Techniques 13
  - 2.3.1 Direct Proof 13
  - 2.3.2 Proof by Contrapositive 14
  - 2.3.3 Proof by Contradiction 14
  - 2.3.4 Mathematical Induction 14
- 2.4 Structural Induction 14

3 Graph Theory and Combinatorics **15**
- 3.1 Graph Definitions 15
- 3.2 Handshaking Lemma 15
- 3.3 Special Graphs 15
- 3.4 Graph Isomorphism 16
- 3.5 Connectivity 16

3.6 Eulerian and Hamiltonian Paths 16
3.7 Trees 17
3.8 Spanning Trees 17
3.9 Planar Graphs 17
3.10 Graph Colouring 17
3.11 Combinatorics 18
3.12 Pigeonhole Principle 18
3.13 Recurrence Relations 18

4 Probability, Automata, and Languages 19
4.1 Probability Fundamentals 19
4.2 Conditional Probability and Bayes 19
4.3 Random Variables and Distributions 19
4.4 Common Distributions 20
4.5 Formal Languages 20
4.6 Regular Expressions and DFA 20
4.7 Context-Free Grammars 21
4.8 Turing Machines 21
4.9 Chomsky Hierarchy 21
4.10 Advanced Computability Topics 21
4.10.1 Church-Turing Thesis 21
4.10.2 Halting Problem 22
4.10.3 Reducibility 22
4.10.4 Complexity Classes 22
4.11 Additional Topics in Set Theory and Logic 23
4.11.1 Naive Set Theory Paradox 23
4.11.2 Boolean Algebra 23
4.12 Advanced Graph Theory 24
4.12.1 Network Flow 24
4.12.2 Shortest Path Algorithms 24
4.12.3 Matching Theory 24
4.13 Generating Functions and Combinatorial Counting 25
4.13.1 Ordinary Generating Functions 25
4.13.2 Exponential Generating Functions 25
4.14 Error-Correcting Codes 25

4.14.1 Linear Codes 25
4.15 Cryptographic Applications 26
4.15.1 Modular Arithmetic 26
4.15.2 Public-Key Cryptography 26
4.16 Expansion: Important Theorems and Proofs 26
4.16.1 Fundamental Theorem of Arithmetic 26
4.16.2 Euclidean Algorithm 27
4.16.3 Chinese Remainder Theorem 27
4.16.4 Stirling's Approximation 27
4.17 Additional Computational Topics 27
4.17.1 NP-Hard and NP-Complete Problems . . . 27
4.17.2 Approximation Algorithms 28
4.17.3 Randomised Algorithms 28
4.18 Combinatorial Designs 28
4.18.1 Latin Squares 28
4.18.2 Orthogonal Latin Squares 29
4.18.3 Block Designs 29
4.19 Matroid Theory 29
4.20 Introduction to Algebra 29
4.20.1 Groups 29
4.20.2 Rings and Fields 30
4.21 Review Questions 30
4.22 Glossary 34
4.23 Further Topics for Advanced Study 36
4.23.1 Measure Theory 36
4.23.2 Topology 36
4.23.3 Category Theory 36
4.23.4 Linear Algebra 36
4.23.5 Numerical Analysis 37
4.23.6 Information Theory 37
4.24 Interdisciplinary Connections 38
4.24.1 Discrete Structures in Physics 38
4.24.2 Discrete Mathematics in Biology 38
4.24.3 Applications to Computer Science 38
4.24.4 Financial Mathematics 38

4.25 Final Thoughts on Mathematical Maturity 39
4.26 Complexity Analysis and Big-O Notation 39
4.26.1 Asymptotic Notation 39
4.26.2 Common Complexity Classes 40
4.26.3 Complexity Analysis Examples 40
4.27 Dynamic Programming 40
4.27.1 Example: Fibonacci with DP 41
4.27.2 Example: Longest Common Subsequence . 41
4.27.3 Example: Knapsack Problem 41
4.28 Greedy Algorithms 42
4.28.1 Activity Selection 42
4.28.2 Huffman Coding 42
4.28.3 Fractional Knapsack 42
4.29 Worked Examples and Problem-Solving Strategies 43
4.29.1 Example 1: Counting with Inclusion-Exclusion 43
4.29.2 Example 2: Recurrence Relation Solution . 43
4.29.3 Example 3: Graph Colouring 43
4.29.4 Example 4: Probability Calculation 44
4.29.5 Example 5: DFA Construction 44
4.29.6 Example 6: Network Flow 44
4.30 Conclusion: Synthesis and Future Directions 45
4.31 Advanced Topics in Logic 45
4.31.1 First-Order Logic (FOL) 45
4.31.2 Resolution and Unification 46
4.31.3 Satisfiability Checking (SAT) 46
4.32 Numerical Methods and Approximation 46
4.32.1 Fixed-Point Iteration 46
4.32.2 Bisection Method 47
4.32.3 Taylor Series Approximation 47
4.33 Stochastic Processes and Markov Chains 47
4.33.1 Markov Chains 47
4.33.2 Poisson Process 47
4.34 Functional Programming Concepts 47
4.34.1 Lambda Calculus 47
4.34.2 Higher-Order Functions 48

4.35 Further Reading and Resources 48
4.36 References 48

Set Theory and Logic

1.1 Set Theory Fundamentals

Definition 1.1. A **set** is an unordered collection of distinct objects (elements). Set membership is denoted $x \in A$ (element x belongs to set A).

Set notation: $A = \{1, 2, 3\}$ or $A = \{x \in \mathbb{Z} : 0 < x \leq 3\}$ (set-builder notation).

Definition 1.2. A **subset** $B \subseteq A$ exists if every element of B is also in A.

Example: $\{1, 2\} \subseteq \{1, 2, 3\}$.

Definition 1.3. The **power set** $P(A)$ is the set of all subsets of A. Its cardinality is $|P(A)| = 2^n$ where $|A| = n$.

Example: $P(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$, with $|P(\{1, 2\})| = 2^2 = 4$ elements.

Definition 1.4. The **Cartesian product** $A \times B$ is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$.

Example: $\{1, 2\} \times \{a, b\} = \{(1, a), (1, b), (2, a), (2, b)\}$.

1.2 Set Operations

Union: $A \cup B = \{x : x \in A \text{ or } x \in B\}$ (elements in either set).

Intersection: $A \cap B = \{x : x \in A \text{ and } x \in B\}$ (elements in both sets).

Complement: $\bar{A} = \{x : x \notin A\}$ (elements not in A).

Difference: $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$ (elements in A but not B).

1.3 Inclusion-Exclusion Principle

For three sets, the principle states:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

This counts elements in any set exactly once by subtracting overcounting from intersections and adding back triple intersection.

Proof Sketch: Elements in exactly one set counted once. Elements in exactly two sets: counted twice in union, once in each pairwise intersection (net: 1). Elements in all three: counted three times in union, three times in pairwise intersections, once in triple (net: 1).

1.4 Relations

Definition 1.5. A **binary relation** on sets A and B is a subset $R \subseteq A \times B$. We write aRb if $(a, b) \in R$.

1.4.1 Relation Properties

Reflexive: aRa for all $a \in A$ (every element relates to itself).

Symmetric: If aRb then bRa (relationship is mutual).

Transitive: If aRb and bRc then aRc (relationship chains).

Antisymmetric: If aRb and bRa then $a = b$ (distinct elements don't relate both ways).

1.5 Equivalence Relations

Definition 1.6. An **equivalence relation** is reflexive, symmetric, and transitive.

Example: Modulo relation $a \equiv b \pmod 3$ (both have same remainder when divided by 3).

Definition 1.7. An **equivalence class** $[a]$ contains all elements equivalent to a: $[a] = \{b : aRb\}$.

For modulo 3: $[1] = \{\ldots, -2, 1, 4, 7, \ldots\}$.

Definition 1.8. A **partition** divides a set into disjoint subsets whose union is the original set.

Equivalence classes form a partition.

1.6 Partial Orders and Hasse Diagrams

Definition 1.9. A **partial order** is reflexive, antisymmetric, and transitive.

Example: Divisibility on $\{1, 2, 3, 6, 12\}$ where $a \leq b$ if a divides b.

Definition 1.10. A **Hasse diagram** visualises partial orders, with a below b if $a < b$ and no element between them.

For divisibility on $\{1, 2, 3, 6, 12\}$: - 1 connects to 2, 3 - 2 connects to 6 - 3 connects to 6 - 6 connects to 12

Definition 1.11. **Minimal element** has no smaller element. **Maximal element** has no larger element.

In the example: 1 is minimal, 12 is maximal.

1.7 Functions

Definition 1.12. A **function** $f : A \to B$ assigns each element $a \in A$ to exactly one element $f(a) \in B$.

Definition 1.13. **Injection (one-to-one):** Different inputs map to different outputs. If $f(a) = f(b)$ then $a = b$.

Definition 1.14. Surjection (onto): Every element in B is mapped to by some element in A. For all $b \in B$, there exists $a \in A$ with $f(a) = b$.

Definition 1.15. Bijection: Both injection and surjection; one-to-one correspondence.

Definition 1.16. Inverse function $f^{-1} : B \to A$ exists only for bijections, with $f^{-1}(f(a)) = a$.

Definition 1.17. Function composition $(f \circ g)(x) = f(g(x))$ applies g first, then f.

1.8 Cardinality

Definition 1.18. Sets have equal **cardinality** if a bijection exists between them.

Definition 1.19. A set is **countable** if bijection exists with $\mathbb{N}$ (natural numbers).

Integers $\mathbb{Z}$ and rationals $\mathbb{Q}$ are countable despite being infinite.

Definition 1.20. A set is **uncountable** if no such bijection exists.

Real numbers $\mathbb{R}$ are uncountable.

Cantor's Diagonal Argument: Proves $|\mathbb{R}| > |\mathbb{N}|$ by assuming list of all reals exists, then constructing a real not on the list (contradiction).

1.9 Propositional Logic

Definition 1.21. A **proposition** is a statement that is either true or false.

Propositions are combined using logical connectives:

Negation: $\neg P$ (not P). Truth value inverted.

Conjunction: $P \wedge Q$ (and). True only if both true.

Disjunction: $P \vee Q$ (or). True if at least one true.

Implication: $P \rightarrow Q$ (if-then). False only when P true and Q false.

Biconditional: $P \leftrightarrow Q$ (if-and-only-if). True when both have same truth value.

1.10 Tautologies and Satisfiability

Definition 1.22. A **tautology** is true under all truth assignments.

Example: $P \vee \neg P$ (law of excluded middle).

Definition 1.23. A **contradiction** is false under all truth assignments.

Example: $P \wedge \neg P$.

Definition 1.24. A formula is **satisfiable** if true under at least one assignment.

1.11 Logical Equivalences

De Morgan's Laws: - $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ - $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

Implication Equivalence: $P \rightarrow Q \equiv \neg P \vee Q$

Biconditional Equivalence: $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

1.12 Normal Forms

Definition 1.25. Conjunctive Normal Form (CNF): Conjunction of disjunctions: $(A \vee B) \wedge (C \vee D)$.

Definition 1.26. Disjunctive Normal Form (DNF): Disjunction of conjunctions: $(A \wedge B) \vee (C \wedge D)$.

Every formula can be converted to CNF or DNF.

Predicate Logic and Proof Techniques

2.1 Predicate Logic Syntax

Definition 2.1. A **predicate** $P(x)$ is a statement parameterised by variables. Truth value depends on variable assignment.

Example: $P(x) =$ "$x > 0$" is true for positive x, false otherwise.

Definition 2.2. Universal quantifier $\forall x : P(x)$ means "$P(x)$ is true for all x".

Definition 2.3. Existential quantifier $\exists x : P(x)$ means "$P(x)$ is true for at least one x".

Example: $\forall x \in \mathbb{R} : x^2 \geq 0$ (true). $\exists x \in \mathbb{R} : x^2 = -1$ (false).

2.2 Valid Inference Rules

Modus Ponens: From P and $P \to Q$, infer Q.

Modus Tollens: From $\neg Q$ and $P \to Q$, infer $\neg P$.

Universal Instantiation: From $\forall x : P(x)$, infer $P(a)$ for specific a.

Existential Generalisation: From $P(a)$, infer $\exists x : P(x)$.

2.3 Proof Techniques

2.3.1 Direct Proof

Assume premises and derive conclusion through logical steps.

Example: Prove $\forall n \in \mathbb{Z}$: if n is even, then n^2 is even.

Proof: If n is even, $n = 2k$ for integer k. Then $n^2 = (2k)^2 = 4k^2 = 2(2k^2)$, which is even.

2.3.2 Proof by Contrapositive

To prove $P \to Q$, prove $\neg Q \to \neg P$.

Example: Prove if n^2 is odd, then n is odd. Contrapositive: if n is even, then n^2 is even (shown above).

2.3.3 Proof by Contradiction

Assume negation of desired conclusion, derive contradiction, conclude original statement.

Example: Prove $\sqrt{2}$ is irrational. Assume $\sqrt{2} = p/q$ in lowest terms. Then $2q^2 = p^2$, so p is even. Let $p = 2r$. Then $2q^2 = 4r^2$, so $q^2 = 2r^2$, so q is even. But if both even, not in lowest terms (contradiction).

2.3.4 Mathematical Induction

Prove $P(n)$ for all natural numbers n: 1. Base case: Prove $P(1)$ true 2. Inductive step: Assume $P(k)$ true, prove $P(k+1)$ true

Example: Prove $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

Base: $\sum_{i=1}^{1} i = 1 = \frac{1 \cdot 2}{2}$ (true).

Step: Assume true for $n = k$. Then $\sum_{i=1}^{k+1} i = \sum_{i=1}^{k} i + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1)+2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$ (true for $k+1$).

2.4 Structural Induction

Induction on recursively-defined structures (lists, trees).

Example: Prove every binary tree with n leaves has $n-1$ internal nodes.

Base: One leaf has zero internal nodes ($1 - 1 = 0$).

Step: Tree with root r and subtrees T_1, T_2. By induction, T_1 has $n_1 - 1$ internal, T_2 has $n_2 - 1$ internal. Combined: $(n_1 - 1) + (n_2 - 1) + 1 = n_1 + n_2 - 1$ internals for $n_1 + n_2$ leaves (true).

Graph Theory and Combinatorics

3.1 Graph Definitions

Definition 3.1. A **graph** $G = (V, E)$ consists of vertices (nodes) V and edges $E \subseteq V \times V$.

Directed: Edges have direction (ordered pairs). **Undirected:** Edges have no direction (unordered pairs).

Weighted: Edges have associated numeric weights.

Multigraph: Multiple edges between same vertices allowed.

Definition 3.2. The **degree** of vertex v is number of incident edges. In directed graphs: in-degree (incoming) and out-degree (outgoing).

3.2 Handshaking Lemma

Theorem 3.1. *For any undirected graph:* $\sum_{v \in V} \deg(v) = 2|E|$.

Proof: Each edge contributes 1 to degree of both endpoints.

3.3 Special Graphs

Complete Graph K_n: Every pair of vertices connected. Has $\binom{n}{2} = \frac{n(n-1)}{2}$ edges.

Bipartite Graph $K_{m,n}$: Vertices divided into two groups; edges only between groups.

Cycle C_n: n vertices in circle. Has n edges.

Path P_n: n vertices in line. Has $n - 1$ edges.

3.4 Graph Isomorphism

Definition 3.3. Graphs G_1 and G_2 are **isomorphic** if bijection $f : V_1 \to V_2$ exists such that $(u, v) \in E_1$ iff $(f(u), f(v)) \in E_2$.

Isomorphic graphs have same structure but may look different.

Invariants under isomorphism include vertex count, edge count, degree sequence.

3.5 Connectivity

Definition 3.4. A graph is **connected** if path exists between every pair of vertices.

Definition 3.5. **Connected component** is maximal connected subgraph.

Definition 3.6. A **bridge** is edge whose removal increases number of connected components.

Definition 3.7. An **articulation point** is vertex whose removal increases components.

3.6 Eulerian and Hamiltonian Paths

Definition 3.8. An **Eulerian path** visits every edge exactly once.

Euler's Theorem: Connected graph has Eulerian circuit iff all vertices have even degree.

Example: Seven Bridges of Königsberg problem requires Eulerian path (solved: no solution as graph has odd-degree vertices).

Definition 3.9. A **Hamiltonian path** visits every vertex exactly once.

No simple characterisation exists. Determining if Hamiltonian path exists is NP-complete.

3.7 Trees

Definition 3.10. A **tree** is connected acyclic graph.

Properties: n vertices, $n-1$ edges, unique path between any two vertices, removing any edge disconnects tree.

3.8 Spanning Trees

Definition 3.11. A **spanning tree** of graph G is subgraph including all vertices and acyclic.

BFS Spanning Tree: Breadth-first search from vertex v produces spanning tree with shortest paths from v.

DFS Spanning Tree: Depth-first search produces spanning tree.

Definition 3.12. **Minimum Spanning Tree (MST)** has minimum total edge weight among all spanning trees.

3.9 Planar Graphs

Definition 3.13. A graph is **planar** if it can be drawn in plane without edge crossings.

Euler's Formula for Planar Graphs: $V-E+F=2$ (vertices minus edges plus faces equals 2).

Theorem 3.2. *Graphs K_5 and $K_{3,3}$ are not planar.*

3.10 Graph Colouring

Definition 3.14. A **graph colouring** assigns colours to vertices such that adjacent vertices have different colours.

Definition 3.15. **Chromatic number** $\chi(G)$ is minimum colours needed.

Four Colour Theorem: Every planar graph is 4-colourable (proven computationally).

Greedy Algorithm: Iteratively colour vertices with smallest available colour. Not always optimal but runs in polynomial time.

3.11 Combinatorics

Permutations: $P(n, r) = \frac{n!}{(n-r)!}$ ordered selections of r items from n.

Combinations: $C(n, r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}$ unordered selections.

Binomial Theorem: $(a + b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k$.

3.12 Pigeonhole Principle

Theorem 3.3. *If $n+1$ pigeons distributed into n holes, at least one hole contains multiple pigeons.*

Application: Among any 13 people, at least two share birth month.

3.13 Recurrence Relations

Definition 3.16. A **recurrence relation** expresses term as function of previous terms.

Example: Fibonacci $F(n) = F(n-1)+F(n-2)$ with $F(1) = 1, F(2) = 1$.

Characteristic Equation Method: For linear homogeneous recurrence, assume $F(n) = r^n$, solve for r, combine solutions.

Probability, Automata, and Languages

4.1 Probability Fundamentals

Definition 4.1. A **probability space** is triple $(\Omega, \mathcal{F}, P)$ where Ω is sample space, $\mathcal{F}$ is event algebra, P is probability measure.

Probability Axioms: - $P(A) \geq 0$ for all events A - $P(\Omega) = 1$ - For disjoint events A, B: $P(A \cup B) = P(A) + P(B)$

4.2 Conditional Probability and Bayes

$$\text{Conditional probability: } P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$\text{Bayes' Theorem: } P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Example: Medical diagnosis. Prior $P(\text{Disease}) = 0.01$. Test accuracy: $P(\text{Positive}|\text{Disease}) = 0.95$, $P(\text{Positive}|\text{No Disease}) = 0.1$.

If test positive: $P(\text{Disease}|\text{Positive}) = \frac{0.95 \cdot 0.01}{0.95 \cdot 0.01 + 0.1 \cdot 0.99} \approx 0.087$ (only 8.7% likely despite positive test).

4.3 Random Variables and Distributions

Definition 4.2. A **random variable** X is function from sample space to real numbers.

Discrete: Probability Mass Function (PMF) $p_X(x) = P(X = x)$.

Expectation: $E[X] = \sum_x x \cdot p_X(x)$.

Variance: $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$.

Continuous: Probability Density Function (PDF) $f_X(x)$, Cumulative Distribution Function (CDF) $F_X(x) = P(X \leq x)$.

4.4 Common Distributions

Bernoulli: Single trial with success probability p. $P(X = 1) = p$.

Binomial: n trials, k successes: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$.

Geometric: Trials until first success: $P(X = k) = (1 - p)^{k-1} p$.

Poisson: Events in fixed interval: $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$.

Uniform: Equally likely on interval $[a, b]$.

Normal: Gaussian with mean μ, standard deviation σ.

4.5 Formal Languages

Definition 4.3. An **alphabet** is finite set of symbols. A **string** is sequence of symbols. A **language** is set of strings.

Language Operations: - **Union:** $L_1 \cup L_2$ (strings in either language) - **Concatenation:** $L_1 \circ L_2 = \{xy : x \in L_1, y \in L_2\}$ - **Kleene Star:** $L^* = \bigcup_{i=0}^{\infty} L^i$ (zero or more repetitions)

4.6 Regular Expressions and DFA

Definition 4.4. A **regular expression** over alphabet Σ matches patterns of strings.

Example: Regex $(a|b)^*ab$ matches strings of a's and b's ending in ab.

Definition 4.5. A **DFA (Deterministic Finite Automaton)** is 5-tuple $(Q, \Sigma, \delta, q_0, F)$: - Q: set of states - Σ: input alphabet - $\delta : Q \times \Sigma \to Q$: transition function - q_0: start state - $F \subseteq Q$: accept states

4.7 Context-Free Grammars

Definition 4.6. A **CFG** is 4-tuple (V, Σ, R, S): - V: set of variables - Σ: set of terminals - R: set of production rules - S: start variable

Example: $S \to aSb|\epsilon$ generates balanced parentheses $\{a^n b^n : n \geq 0\}$.

Definition 4.7. Chomsky Normal Form restricts rules to $A \to BC$ or $A \to a$.

4.8 Turing Machines

Definition 4.8. A **Turing Machine** is 7-tuple with finite control, infinite tape, and read-write head.

TMs formalise computation; problems solvable by TM are computable.

Definition 4.9. A language is **decidable** if TM exists that halts on all inputs, accepting exactly the language.

4.9 Chomsky Hierarchy

- Type 0: Unrestricted; recognised by Turing Machine - Type 1: Context-sensitive; recognised by linear-bounded automaton - Type 2: Context-free; recognised by pushdown automaton - Type 3: Regular; recognised by finite automaton

Each level strictly contained in previous (Type 3 Type 2 Type 1 Type 0).

4.10 Advanced Computability Topics

4.10.1 Church-Turing Thesis

The Church-Turing Thesis states that a function is computable by algorithm if and only if it is computable by Turing Machine. This

connects informal computation with formal models.

Consequences:

- Any reasonable model of computation is equivalent to TM
- No model more powerful than TM (within Church-Turing framework)
- Defines boundaries of computability

4.10.2 Halting Problem

Theorem 4.1. *No algorithm exists that, given arbitrary program and input, determines whether the program terminates.*

Proof (by contradiction): Assume H solves halting problem. Define D that calls $H(D, D)$ and does opposite of H's answer. Then $D(D)$ leads to contradiction.

Consequence: Some problems are uncomputable; no universal algorithm exists for all problems.

4.10.3 Reducibility

Definition 4.10. Problem A **reduces to** problem B (written $A \leq_m B$) if solution to B can be used to solve A.

If A reduces to B and B is decidable, then A is decidable. Contrapositive: if A is undecidable, then B is undecidable.

4.10.4 Complexity Classes

Definition 4.11. **P** is class of problems decidable in polynomial time.

Definition 4.12. **NP** is class of problems verifiable in polynomial time.

P vs NP: Does $P = NP$? Major open problem in computer science. Most believe $P \neq NP$ (some problems harder to solve than verify).

NP-Complete: Hardest problems in NP; any NP problem reduces to NP-complete problem.

Examples: - SAT: Satisfiability of Boolean formulas - Hamiltonian Path - Clique: Find clique of size k in graph - Travelling Salesman Problem

4.11 Additional Topics in Set Theory and Logic

4.11.1 Naive Set Theory Paradox

Naive set theory allows any property to define set: $A = \{x : P(x)\}$.

Russell's Paradox: Let $R = \{x : x \notin x\}$ (set of all sets not containing themselves).

Is $R \in R$? If yes, then $R \notin R$ (contradiction). If no, then $R \in R$ (contradiction).

Resolution: Formal set theory (ZFC) restricts set formation.

4.11.2 Boolean Algebra

Definition 4.13. Boolean algebra is algebraic structure with operations AND, OR, NOT satisfying certain axioms.

Laws:

- Idempotence: $A \wedge A = A$, $A \vee A = A$
- Absorption: $A \vee (A \wedge B) = A$
- Distributivity: $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$
- Complement: $A \vee \neg A = \top$, $A \wedge \neg A = \bot$

Boolean algebras formalise digital logic and switch design.

4.12 Advanced Graph Theory

4.12.1 Network Flow

Definition 4.14. Flow network is directed weighted graph with source s and sink t.

Each edge has capacity. Flow assignment respects capacity constraints and conservation (inflow = outflow at each intermediate node).

Max-Flow Min-Cut Theorem: Maximum flow equals minimum cut capacity.

Algorithms: Ford-Fulkerson, Edmonds-Karp.

4.12.2 Shortest Path Algorithms

Dijkstra's Algorithm: Finds shortest path from source to all vertices in non-negative weighted graph. Complexity: $O((V+E)\log V)$ with binary heap.

Bellman-Ford: Handles negative weights; detects negative cycles. Complexity: $O(VE)$.

Floyd-Warshall: All-pairs shortest paths. Complexity: $O(V^3)$.

4.12.3 Matching Theory

Definition 4.15. A **matching** in graph is set of edges without common vertices.

Maximum Matching: Matching with most edges.

Hall's Marriage Theorem: Bipartite graph has perfect matching iff for every subset S of one part, $|N(S)| \geq |S|$ (neighbours at least as many as S).

4.13 Generating Functions and Combinatorial Counting

4.13.1 Ordinary Generating Functions

Definition 4.16. **OGF** of sequence $a_0, a_1, a_2, \ldots$ is $\sum_{n=0}^{\infty} a_n x^n$.

Example: OGF of Fibonacci (F_n): $F(x) = \frac{x}{1-x-x^2}$.

Generating functions transform recurrences into polynomial equations.

4.13.2 Exponential Generating Functions

Definition 4.17. **EGF** is $\sum_{n=0}^{\infty} a_n \frac{x^n}{n!}$.

EGF useful for counting permutations and labelled structures.

4.14 Error-Correcting Codes

Definition 4.18. **Code** is set of codewords; **distance** is minimum difference between codewords.

Hamming Distance: Number of positions differing between strings.

Hamming Bound: Code with distance d corrects up to $\lfloor (d-1)/2 \rfloor$ errors.

4.14.1 Linear Codes

Hamming Code: Detects and corrects single-bit errors. Adds parity check bits.

Reed-Solomon Code: Used in QR codes, data storage. Corrects multiple errors.

Codes balance redundancy (overhead) vs error correction capability.

4.15 Cryptographic Applications

4.15.1 Modular Arithmetic

Definition 4.19. $a \equiv b \pmod{n}$ if $n|(a-b)$ (reads "a congruent to b modulo n").

Properties:

- $(a+b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
- $(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n$

Fermat's Little Theorem: If p prime and $\gcd(a,p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.

Euler's Theorem: If $\gcd(a,n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod{n}$ where $\phi(n)$ is Euler's totient function.

4.15.2 Public-Key Cryptography

RSA: Based on difficulty of factoring. Public key (n,e), private key d.

Encryption: $C = M^e \bmod n$. Decryption: $M = C^d \bmod n$.

Security: Factoring $n = pq$ is hard for large primes.

Discrete Logarithm Problem: Given a, b, p, find x such that $a^x \equiv b \pmod{p}$.

Believed hard; forms basis of Diffie-Hellman key exchange and elliptic curve cryptography.

4.16 Expansion: Important Theorems and Proofs

4.16.1 Fundamental Theorem of Arithmetic

Theorem 4.2. *Every integer $n > 1$ uniquely factors into primes (up to ordering): $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$.*

Proof sketch: Existence by induction. Uniqueness by contradiction using prime properties.

4.16.2 Euclidean Algorithm

Computes $\gcd(a, b)$ efficiently:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Terminates when remainder is zero. Complexity: $O(\log \min(a, b))$.

Extended Euclidean Algorithm finds integers x, y such that $ax + by = \gcd(a, b)$.

4.16.3 Chinese Remainder Theorem

Theorem 4.3. *System of congruences $x \equiv a_i \pmod{m_i}$ for coprime m_i has unique solution modulo $\prod m_i$.*

Applications: Efficient modular arithmetic in cryptography.

4.16.4 Stirling's Approximation

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Provides good approximation for large n; useful in complexity analysis.

4.17 Additional Computational Topics

4.17.1 NP-Hard and NP-Complete Problems

NP-Hard: Every NP problem reduces to this problem (but problem itself may not be in NP).

NP-Complete: NP-Hard AND in NP.

Important NP-Complete problems:

- Boolean Satisfiability (SAT)
- 3-SAT: SAT with at most 3 literals per clause
- Clique: Find largest complete subgraph

- Vertex Cover: Minimum set of vertices covering all edges
- Travelling Salesman Problem
- Knapsack: Pack items maximising value with weight constraint
- Subset Sum: Find subset summing to target

No polynomial algorithm known; widely believed impossible.

4.17.2 Approximation Algorithms

For NP-hard problems, polynomial approximation algorithms provide solutions within guaranteed factor of optimal.

Vertex Cover Approximation: Greedy algorithm achieves 2-approximation (solution within 2× optimal).

Travelling Salesman Approximation: Christofides algorithm achieves 1.5-approximation for metric TSP.

4.17.3 Randomised Algorithms

Use randomness to solve problems efficiently.

QuickSort: Average $O(n \log n)$ despite worst-case $O(n^2)$ (randomised pivot selection).

Miller-Rabin Primality Test: Probabilistic polynomial-time primality test. Can fail with controlled probability.

Las Vegas Algorithms: Always correct; randomised runtime.

Monte Carlo Algorithms: Randomised correctness; bounded error probability.

4.18 Combinatorial Designs

4.18.1 Latin Squares

$n \times n$ grid filled with n symbols such that each symbol appears exactly once per row and column.

Application: Experimental design (balanced experiments).

4.18.2 Orthogonal Latin Squares

Two Latin squares are orthogonal if overlaying them produces all n^2 pairs.

Euler's Conjecture (Disproven): No orthogonal Latin squares of side $4k + 2$ exist. (Disproven for most cases in 1960s.)

4.18.3 Block Designs

Definition 4.20. (v, k, λ)**-block design** partitions v points into blocks of size k such that each pair appears in exactly λ blocks.

Application: Statistical experimental designs balancing confounding factors.

4.19 Matroid Theory

Definition 4.21. A **matroid** $(S, \mathcal{I})$ consists of finite set S and collection $\mathcal{I}$ of independent sets satisfying: - $\emptyset \in \mathcal{I}$ - Downward closed: if $A \in \mathcal{I}$ and $B \subseteq A$ then $B \in \mathcal{I}$ - Exchange property: if $A, B \in \mathcal{I}$ and $|A| < |B|$, then $\exists x \in B \setminus A$ with $A \cup \{x\} \in \mathcal{I}$

Matroids generalise linear independence. Greedy algorithm finds maximum independent set in matroids.

4.20 Introduction to Algebra

4.20.1 Groups

Definition 4.22. A **group** $(G, \cdot)$ is set with binary operation satisfying: - Closure: $a \cdot b \in G$ for all $a, b \in G$ - Associativity: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ - Identity: $\exists e \in G$ with $a \cdot e = e \cdot a = a$ - Inverse: For each $a \in G$, $\exists a^{-1}$ with $a \cdot a^{-1} = e$

Example: $(\mathbb{Z}, +)$ with identity 0 and inverse of a being $-a$.

4.20.2 Rings and Fields

Definition 4.23. A **ring** has two operations $(+, \cdot)$ with addition forming group, multiplication associative, and distributive law.

Definition 4.24. A **field** is commutative ring where every non-zero element has multiplicative inverse.

Examples: $\mathbb{Q}, \mathbb{R}, \mathbb{C}$, and $\mathbb{Z}_p$ (integers modulo prime p).

4.21 Review Questions

1. Define set and list notation for $\{x \in \mathbb{Z} : x^2 < 10\}$
2. Calculate cardinality of power set of 4-element set
3. Compute $|A \cup B|$ given $|A| = 5$, $|B| = 3$, $|A \cap B| = 1$
4. Explain subset notation; distinguish $\subseteq$ from $\in$
5. Define and provide example of binary relation
6. What properties characterise equivalence relation?
7. Explain equivalence class and partition
8. Define partial order and give example
9. Describe Hasse diagram and its construction
10. Define function, injection, surjection, bijection
11. Prove $f(x) = 2x$ is bijection from $\mathbb{Z}$ to even integers
12. Explain cardinality and countability
13. Describe Cantor's diagonal argument
14. Why is $|\mathbb{R}| > |\mathbb{N}|$?
15. Write truth table for $P \wedge (Q \vee \neg R)$

16. Distinguish tautology from satisfiable formula
17. State De Morgan's laws with example
18. Convert $\neg(P \vee Q) \wedge R$ to CNF
19. Define predicate logic and quantifiers
20. Apply modus ponens to infer conclusion
21. Provide direct proof: if n even then n^2 even
22. Explain proof by contrapositive
23. Outline proof by contradiction for $\sqrt{3}$ irrational
24. Apply mathematical induction to prove sum formula
25. Distinguish weak from strong induction
26. Define structural induction on trees
27. Draw graph with degree sequence $(2, 3, 1, 2)$
28. Verify handshaking lemma for K_4
29. List edges in complete bipartite graph $K_{2,3}$
30. Define graph isomorphism and find isomorphic pair
31. Distinguish bridge from articulation point
32. State Euler's theorem for Eulerian paths
33. Explain why Hamiltonian path problem is hard
34. Define tree and prove n vertices $\Rightarrow n - 1$ edges
35. Distinguish spanning tree from minimum spanning tree
36. State Euler's formula for planar graphs
37. Explain why K_5 is not planar

38. Define graph colouring and chromatic number
39. Calculate $P(5, 2)$ and $C(5, 2)$
40. Explain NP-completeness and provide example problems
41. What is an approximation algorithm and its guarantee?
42. Distinguish Las Vegas from Monte Carlo algorithms
43. Define Latin square and its properties
44. What is orthogonal Latin square?
45. Explain block design and its applications
46. Define matroid and independence property
47. Describe group, ring, and field algebraic structures
48. What is the identity element in $(\mathbb{Z}, +)$?
49. Explain why $\mathbb{Z}_p$ forms a field for prime p
50. Compare Church-Turing thesis with Turing machine definition
51. Prove why halting problem is undecidable
52. Define problem reduction in computability
53. What is relationship between P and NP classes?
54. Explain SAT problem and its NP-completeness
55. How does Dijkstra's algorithm find shortest paths?
56. Describe Bellman-Ford algorithm advantages over Dijkstra
57. Apply Floyd-Warshall for all-pairs shortest paths
58. State Hall's Marriage Theorem

59. Explain generating functions for Fibonacci sequence
60. What is difference between OGF and EGF?
61. Define Hamming distance for error-correcting codes
62. How many errors can Hamming code correct?
63. Describe Reed-Solomon codes and applications
64. Define modular arithmetic and congruence
65. State and apply Fermat's Little Theorem
66. Explain Euler's totient function $\phi(n)$
67. How does RSA encryption work?
68. What is difficulty of discrete logarithm problem?
69. Prove Fundamental Theorem of Arithmetic
70. Describe Euclidean algorithm for GCD
71. Apply Extended Euclidean algorithm
72. State and apply Chinese Remainder Theorem
73. Explain Stirling's approximation for factorials
74. Apply binomial theorem to expand $(x + y)^3$
75. State pigeonhole principle and provide application
76. Solve Fibonacci recurrence using characteristic equation
77. Define probability space and axioms
78. Apply Bayes' theorem to medical diagnosis
79. Distinguish discrete from continuous random variables
80. Calculate expectation and variance of Bernoulli distribution

81. Define language over alphabet
82. Convert regex to corresponding DFA
83. Explain difference between DFA and NFA
84. Define CFG and provide grammar for $a^n b^n$
85. List Chomsky hierarchy levels and example languages

4.22 Glossary

1. **Bijection:** One-to-one correspondence function
2. **Cardinality:** Size of set; count of elements
3. **CNF:** Conjunctive Normal Form; conjunction of disjunctions
4. **Countable:** Set with bijection to natural numbers
5. **DFA:** Deterministic Finite Automaton for regular languages
6. **DNF:** Disjunctive Normal Form; disjunction of conjunctions
7. **Equivalence Relation:** Reflexive, symmetric, transitive relation
8. **Function:** Maps input domain to output codomain
9. **Graph:** Set of vertices and edges representing connections
10. **Injection:** One-to-one function; injective
11. **Isomorphic:** Same structure; bijection preserving relations
12. **Language:** Set of strings over alphabet
13. **Partial Order:** Reflexive, antisymmetric, transitive relation
14. **PMF:** Probability Mass Function for discrete random variable

15. **Power Set:** Set of all subsets; cardinality 2^n
16. **Probability:** Measure of likelihood between 0 and 1
17. **Recurrence:** Relation expressing term from previous terms
18. **Set:** Unordered collection of distinct elements
19. **Surjection:** Onto function; surjective
20. **Tree:** Connected acyclic graph
21. **Turing Machine:** Abstract model of computation
22. **Undecidable:** Problem not solvable by any algorithm
23. **Bijection:** Function that is both injection and surjection
24. **Congruence:** Equivalence modulo an integer
25. **Field:** Algebraic structure with addition and multiplication
26. **Flow Network:** Directed graph with edge capacities
27. **Group:** Algebraic structure with associative operation
28. **Matroid:** Generalisation of linear independence
29. **NP-Complete:** Hardest problems in NP class
30. **Primality:** Property of being divisible only by 1 and itself
31. **Reduction:** Using solution of problem to solve another
32. **Ring:** Algebraic structure with addition and multiplication
33. **Totient Function:** Count of integers coprime to n
34. **Big-O Notation:** Asymptotic upper bound on function growth
35. **Big-Omega:** Asymptotic lower bound on function growth
36. **Big-Theta:** Asymptotic tight bound on function growth

37. **Memoization:** Caching results to avoid recomputation
38. **Greedy Algorithm:** Makes locally optimal choices
39. **Dynamic Programming:** Solves overlapping subproblems with memoisation
40. **Fixed-Point:** Solution to equation $x = g(x)$
41. **Markov Chain:** Stochastic process with memoryless property
42. **Lambda Calculus:** Formal system of function definition

4.23 Appendix: Quick Reference

4.23.1 Permutations and Combinations

$P(n, r) = \frac{n!}{(n-r)!}$ (ordered arrangements)
$C(n, r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}$ (unordered selections)

4.23.2 Boolean Algebra Laws

De Morgan: $\neg(A \wedge B) = \neg A \vee \neg B$
Distributive: $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$

4.23.3 Complexity Classes (Fast to Slow)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

4.23.4 Commonly Used Formulas

Sum of first n integers: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
Sum of geometric series: $\sum_{i=0}^{n-1} r^i = \frac{r^n-1}{r-1}$ (for $r \neq 1$)
Binomial expansion: $(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k$

4.24 Further Topics for Advanced Study

4.24.1 Measure Theory

Measure theory formalises the notion of size for sets beyond cardinality.

Definition 4.25. A **measure** μ assigns non-negative real numbers to sets satisfying countable additivity: $\mu(\bigcup_i A_i) = \sum_i \mu(A_i)$ for disjoint A_i.

Lebesgue measure on real line extends concept of length. Enables rigorous probability theory and integration.

4.24.2 Topology

Topology studies properties preserved under continuous deformations.

Definition 4.26. A **topological space** is pair (X, τ) where τ is collection of open sets satisfying: - $\emptyset, X \in \tau$ - Unions of open sets are open - Finite intersections of open sets are open

Key concepts: continuity, compactness, connectedness.

4.24.3 Category Theory

Definition 4.27. A **category** consists of objects and morphisms (arrows) between them, with composition rule.

Provides unified framework for studying mathematical structures. Used in algebra, topology, logic, and computer science.

4.24.4 Linear Algebra

Definition 4.28. A **vector space** over field F is set V with addition and scalar multiplication satisfying vector space axioms.

Basis: Minimal spanning set. All bases have same cardinality (dimension).

Linear Transformation: Function preserving vector space structure.

Matrix Representation: Linear transformations represented as matrices.

Eigenvalues and Eigenvectors: For matrix A, vector v is eigenvector with eigenvalue λ if $Av = \lambda v$.

Applications: Markov chains, graphics transformations, data analysis (PCA).

4.24.5 Numerical Analysis

Studies algorithms for solving mathematical problems numerically.

Root Finding: Newton's method finds roots iteratively: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.

Numerical Integration: Simpson's rule, trapezoidal rule approximate integrals.

Linear Systems: Gaussian elimination solves $Ax = b$.

Error Analysis: Quantifies rounding and truncation errors.

4.24.6 Information Theory

Definition 4.29. Shannon Entropy $H(X) = -\sum p(x) \log_2 p(x)$ measures uncertainty in random variable.

Mutual Information: Measures dependence between random variables.

Channel Capacity: Maximum information rate through noisy channel.

Huffman Coding: Optimal compression scheme using variable-length codes.

4.25 Interdisciplinary Connections

4.25.1 Discrete Structures in Physics

Quantum computing uses quantum bits (qubits) and quantum gates, redefining computation fundamentals.

Graph Theory in Physics: Feynman diagrams represent particle interactions as graphs.

4.25.2 Discrete Mathematics in Biology

DNA Sequences: Strings over 4-letter alphabet $\{A, T, G, C\}$. Sequence alignment via edit distance.

Phylogenetic Trees: Trees representing evolutionary relationships.

Protein Folding: Complex optimisation problem on graph of conformations.

4.25.3 Applications to Computer Science

Compiler Design: Parsing uses finite automata and context-free grammars.

Database Theory: Relational algebra formalises query operations.

Cryptography: Number theory provides security foundations.

Algorithm Design: Graph algorithms, dynamic programming, greedy algorithms.

4.25.4 Financial Mathematics

Probability Models: Geometric Brownian motion models stock prices.

Stochastic Processes: Markov chains model transition probabilities.

Optimisation: Portfolio optimisation balances risk and return.

4.26 Final Thoughts on Mathematical Maturity

Understanding discrete mathematics and mathematical foundations of computer science requires:

- **Rigorous Thinking:** Precise definitions, logical inference
- **Problem-Solving:** Multiple approaches to challenges
- **Proof Writing:** Clear, structured mathematical arguments
- **Abstraction:** Generalising from specific to universal principles
- **Intuition:** Building geometric and conceptual understanding

Mastery develops through practice: solving problems, reading proofs, writing own arguments. Success requires patience with abstraction and persistence through difficult concepts.

The mathematical foundations covered in this book enable understanding of modern computer science algorithms, complexity analysis, cryptography, and formal verification. Continuous engagement with mathematical thinking strengthens analytical skills applicable far beyond computer science.

4.27 Complexity Analysis and Big-O Notation

4.27.1 Asymptotic Notation

Definition 4.30. Big-O notation $O(g(n))$ is set of functions $f(n)$ such that $\exists c > 0, n_0 > 0$ with $f(n) \leq c \cdot g(n)$ for all $n > n_0$.

Intuition: $f(n)$ grows at most as fast as $g(n)$.

Example: $3n^2+5n+100 = O(n^2)$ (constant factors and lower-order terms ignored).

Definition 4.31. Big-Omega $\Omega(g(n))$: $f(n) \geq c \cdot g(n)$ (lower bound).

Definition 4.32. Big-Theta $\Theta(g(n))$: $f(n)$ between $c_1 g(n)$ and $c_2 g(n)$ (tight bound).

4.27.2 Common Complexity Classes

From fastest to slowest:

$O(1)$**:** Constant time (hash table lookup).
$O(\log n)$**:** Logarithmic (binary search).
$O(n)$**:** Linear (simple loop).
$O(n \log n)$**:** Linearithmic (merge sort).
$O(n^2)$**:** Quadratic (bubble sort, nested loops).
$O(n^3)$**:** Cubic (matrix multiplication naive).
$O(2^n)$**:** Exponential (brute force subsets).
$O(n!)$**:** Factorial (permutations).

4.27.3 Complexity Analysis Examples

Linear Search: Iterates through array checking each element. Worst case: $O(n)$ (element at end or absent).

Binary Search: Halves search space each iteration. Complexity: $O(\log n)$.

Merge Sort: Divide-and-conquer strategy. Recurrence: $T(n) = 2T(n/2) + n \Rightarrow T(n) = O(n \log n)$.

Quick Sort: Average $O(n \log n)$, worst $O(n^2)$ (poor pivot choices).

Bubble Sort: Compares adjacent elements; swaps if out of order. Complexity: $O(n^2)$.

4.28 Dynamic Programming

Definition 4.33. Dynamic Programming solves problems by breaking into overlapping subproblems, computing solutions once and reusing.

Key characteristics:

- Optimal substructure: Optimal solution includes optimal solutions to subproblems
- Overlapping subproblems: Subproblems recur multiple times

4.28.1 Example: Fibonacci with DP

Naive recursive: $F(n) = F(n-1) + F(n-2)$ recalculates same values exponentially $O(2^n)$.

DP approach (memoization):

```
cache = {}
def fib(n):
    if n in cache: return cache[n]
    if n <= 1: result = n
    else: result = fib(n-1) + fib(n-2)
    cache[n] = result
    return result
```

Complexity: $O(n)$ (each value computed once).

4.28.2 Example: Longest Common Subsequence

Problem: Find longest subsequence common to two strings.

DP solution: - $dp[i][j]$ = LCS length for first i characters of string 1 and first j of string 2 - Recurrence: If $s_1[i] = s_2[j]$: $dp[i][j] = 1 + dp[i-1][j-1]$ - Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

Complexity: $O(m \cdot n)$ where m, n are string lengths.

4.28.3 Example: Knapsack Problem

Problem: Pack items maximising value with weight constraint.

DP solution: - $dp[w]$ = maximum value achievable with weight limit w - For each item: $dp[w] = \max(dp[w], dp[w - \text{weight}] + \text{value})$

Complexity: $O(n \cdot W)$ where n is item count, W is weight limit.

4.29 Greedy Algorithms

Greedy algorithms make locally optimal choices hoping to reach global optimum.

4.29.1 Activity Selection

Problem: Select maximum compatible activities.

Greedy approach: Always select activity finishing earliest.

Proof: Optimal solution exists including earliest-finishing activity. Replace any earlier activity with it.

Complexity: $O(n \log n)$ (sorting by finish time).

4.29.2 Huffman Coding

Problem: Assign variable-length codes minimising expected length.

Greedy approach: Repeatedly combine two lowest-frequency characters.

Result: Optimal prefix-free code.

Complexity: $O(n \log n)$.

4.29.3 Fractional Knapsack

Problem: Pack fractional items maximising value.

Greedy approach: Select items by value-to-weight ratio (highest first).

This is optimal for fractional version (not integral knapsack).

4.30 Worked Examples and Problem-Solving Strategies

4.30.1 Example 1: Counting with Inclusion-Exclusion

Problem: How many integers from 1 to 100 are divisible by 2, 3, or 5?

Solution: Let A = divisible by 2, B = divisible by 3, C = divisible by 5.

$|A| = \lfloor 100/2 \rfloor = 50$
$|B| = \lfloor 100/3 \rfloor = 33$
$|C| = \lfloor 100/5 \rfloor = 20$
$|A \cap B| = \lfloor 100/6 \rfloor = 16$
$|A \cap C| = \lfloor 100/10 \rfloor = 10$
$|B \cap C| = \lfloor 100/15 \rfloor = 6$
$|A \cap B \cap C| = \lfloor 100/30 \rfloor = 3$
By inclusion-exclusion: $50 + 33 + 20 - 16 - 10 - 6 + 3 = 74$.

4.30.2 Example 2: Recurrence Relation Solution

Problem: Solve $a_n = 3a_{n-1} - 2a_{n-2}$ with $a_0 = 1, a_1 = 3$.

Solution: Characteristic equation: $r^2 - 3r + 2 = 0 \Rightarrow (r - 1)(r - 2) = 0 \Rightarrow r \in \{1, 2\}$.

General solution: $a_n = c_1 \cdot 1^n + c_2 \cdot 2^n = c_1 + c_2 \cdot 2^n$.

Using initial conditions: $a_0 = c_1 + c_2 = 1$ and $a_1 = c_1 + 2c_2 = 3$.

Solving: $c_2 = 2, c_1 = -1$.

Therefore: $a_n = -1 + 2 \cdot 2^n = 2^{n+1} - 1$.

Verification: $a_0 = 2 - 1 = 1$, $a_1 = 4 - 1 = 3$.

4.30.3 Example 3: Graph Colouring

Problem: Colour graph with vertices $\{A, B, C, D\}$ and edges $\{(A, B), (B, C), (C,$

Solution: Use greedy algorithm: - Colour A with 1 - Colour B with 2 (adjacent to A) - Colour C with 3 (adjacent to A, B) - Colour D with 2 (adjacent to A, C only; can reuse 2)

Result: $\chi(G) = 3$ (colours needed).

4.30.4 Example 4: Probability Calculation

Problem: Two dice rolled. What is probability that sum is at least 10?

Solution: Total outcomes: $6 \times 6 = 36$.

Favourable outcomes: $(4, 6), (5, 5), (5, 6), (6, 4), (6, 5), (6, 6)$ — total 6 outcomes.

Probability: $6/36 = 1/6 \approx 0.167$.

4.30.5 Example 5: DFA Construction

Problem: Construct DFA accepting binary strings ending in "01".

Solution: - State q_0: Start (no progress) - State q_1: Just read "0" - State q_2: Read "01" (accept)

Transitions: - $q_0 \xrightarrow{0} q_1$, $q_0 \xrightarrow{1} q_0$ - $q_1 \xrightarrow{0} q_1$, $q_1 \xrightarrow{1} q_2$ - $q_2 \xrightarrow{0} q_1$, $q_2 \xrightarrow{1} q_0$

Accept states: $\{q_2\}$.

4.30.6 Example 6: Network Flow

Problem: Find maximum flow from source s to sink t in network with edge capacities.

Network: $s \to a$ (capacity 10), $a \to t$ (capacity 5), $s \to b$ (capacity 8), $b \to t$ (capacity 7), $a \to b$ (capacity 3).

Solution: Using Ford-Fulkerson: - Augment along $s \to a \to t$ (capacity 5) - Augment along $s \to b \to t$ (capacity 7) - Total flow: 12

Maximum flow = 12 (limited by bottleneck capacities).

4.31 Conclusion: Synthesis and Future Directions

The mathematical foundations covered in this book form the bedrock of modern computer science. From set theory's formal definitions to complexity theory's performance analysis, these concepts appear throughout computing.

Key takeaways:

- **Rigour:** Mathematics provides precise frameworks for reasoning about computation
- **Abstraction:** High-level thinking abstracts away implementation details
- **Algorithms:** Mathematical analysis guides algorithm design and optimisation
- **Limits:** Computability theory reveals what is fundamentally impossible
- **Applications:** Cryptography, coding theory, AI all rest on mathematical foundations

As technology advances, understanding these foundations becomes increasingly important for navigating emerging challenges in quantum computing, machine learning, and distributed systems.

Continuous learning and engagement with mathematical thinking strengthens the analytical skills essential to computer science success.

4.32 Advanced Topics in Logic

4.32.1 First-Order Logic (FOL)

Definition 4.34. First-Order Logic extends propositional logic with predicates and quantification over objects.

Syntax includes:

- Terms: constants, variables, function applications
- Formulas: atomic formulas, connectives, quantifiers

Example: $\forall x : \text{Student}(x) \rightarrow \text{Learns}(x, \text{Math})$ (all students learn math).

4.32.2 Resolution and Unification

Resolution Principle: Inference rule for automated theorem proving.

From clauses $(P \vee Q)$ and $(\neg P \vee R)$, infer $(Q \vee R)$ (resolvent).

Unification: Algorithm for finding substitutions making terms identical.

Example: Unify $P(x, f(y))$ and $P(a, f(b))$ by substitution $x \mapsto a, y \mapsto b$.

4.32.3 Satisfiability Checking (SAT)

SAT solvers determine if propositional formula is satisfiable.

DPLL Algorithm: Systematic search with unit propagation and pure literal elimination.

Conflict-Driven Learning: Modern SAT solvers learn from conflicts, enabling exponential speedups.

Applications: Verification, planning, constraint satisfaction.

4.33 Numerical Methods and Approximation

4.33.1 Fixed-Point Iteration

To solve $f(x) = 0$, rewrite as $x = g(x)$ and iterate $x_{n+1} = g(x_n)$.

Convergence depends on $|g'(x)| < 1$ in neighbourhood of fixed point.

4.33.2 Bisection Method

For continuous f with $f(a) < 0 < f(b)$, repeatedly bisect interval.

Guaranteed convergence; slow ($O(\log(1/\epsilon))$ iterations for accuracy ϵ).

4.33.3 Taylor Series Approximation

$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$

Enables approximating transcendental functions and solving differential equations.

4.34 Stochastic Processes and Markov Chains

4.34.1 Markov Chains

Definition 4.35. Markov Chain is sequence of random variables where future depends only on current state (memoryless property).

Transition Matrix P with $P_{ij} = P(X_{n+1} = j | X_n = i)$.

Stationary Distribution π satisfies $\pi P = \pi$ and $\sum_i \pi_i = 1$.

Applications: Page ranking (Google), queueing systems, hidden Markov models.

4.34.2 Poisson Process

Models random arrival events at constant average rate.

Number of arrivals in time interval t follows Poisson distribution with parameter λt.

Applications: Network traffic, radioactive decay, customer arrivals.

4.35 Functional Programming Concepts

4.35.1 Lambda Calculus

Formal system of function definition and application.

Syntax: Variables x, abstractions $\lambda x.M$, applications MN.

Evaluation: $(\lambda x.M)N \to M[N/x]$ (substitution).

Foundation for functional programming languages and proof systems.

4.35.2 Higher-Order Functions

Functions that take or return functions.

Map: Apply function to each element: map$(f, [1, 2, 3]) = [f(1), f(2), f(3)]$

Filter: Keep elements satisfying predicate: filter$(p, [1, 2, 3, 4]) = [2, 4]$ for $p =$ even.

Fold/Reduce: Accumulate values: fold$(+, [1, 2, 3, 4], 0) = 10$.

Enables expressive, concise functional code.

4.36 Further Reading and Resources

For deepening understanding of mathematical foundations:

Textbooks: - Rosen: Discrete Mathematics and Its Applications - Sipser: Theory of Computation - Cormen, Leiserson, Rivest, Stein: Introduction to Algorithms

Online Resources: - Khan Academy: Mathematics courses - MIT OpenCourseWare: Computer science courses - Project Euler: Programming challenges

Practice: - Solve problems from textbooks systematically - Participate in programming competitions (ACM-ICPC, Codeforces) - Read research papers in areas of interest

Building expertise requires sustained engagement with challenging material and practical application of concepts to real problems.

4.37 References

1. Rosen, K. H. (2019). Discrete Mathematics and Its Applications (8th ed.). McGraw-Hill.

2. Sipser, M. (2012). Introduction to the Theory of Computation (3rd ed.). Cengage.

3. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Pearson.

4. Graham, R. L., Knuth, D. E., & Patashnik, O. (1994). Concrete Mathematics (2nd ed.). Addison-Wesley.

5. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

6. Hammack, R. (2018). Book of Proof (3rd ed.). Virginia Commonwealth University.

7. Bondy, J. A., & Murty, U. S. R. (2008). Graph Theory. Springer.

8. Ross, S. M. (2014). A First Course in Probability (9th ed.). Pearson.